



Short and Winding Road: Software in Car Navigation Systems

Han Schaminée and Hans Aerts

Car navigation started as an embedded product in expensive cars. Then it became an independent box, sold aftermarket. The volume that companies like TomTom have been able to accumulate now allows them to pursue the automotive original equipment manufacturers market. —*Michiel van Genuchten and Les Hatton*

TOMTOM IS KNOWN for *aftermarket* navigation systems—separate devices sold independently of the car. In April 2009, TomTom launched its first in-dash navigation system (built into the dashboard by the car manufacturer at the factory). The system is upgraded annually, allowing the introduction of new features and innovations over its lifetime. This column describes the product, its place in the automotive industry, and the development of its software.

A Short History of Car Navigation

Car navigation started as a feature of expensive cars in the 1980s. The systems didn't use GPS and each car manufacturer's system was unique. It became a separate aftermarket device about 10 years ago, when the US made a more accurate GPS signal available for civilian use, thus making navigation less dependent on the vehicle itself. Today, TomTom's sales exceed 10 million devices per year, which is more than the two to nine million cars that the largest car manufacturers sell each year. This lets TomTom invest more in car navigation than the typical car original equipment manufacturer (OEM). Thus, TomTom can introduce new functionalities

faster when the OEMs agree to use the synergy of aftermarket products.

Handling Different Innovation Speeds

Over the last two decades, the automotive industry has gone through an impressive turnaround to ensure substantially higher quality levels. An example is the requirement to have less than a 0.01 percent failure rate at the car manufacturing line. A car consists of a large number of subassemblies from many different suppliers. The industry introduced rigid processes for architecture, validation, and supplier management to realize this breakthrough in quality. Better quality had been a differentiator in the market for a long time, but most car manufacturers have now achieved the same high levels. Customer dissatisfaction no longer comes primarily from quality issues, but more from slow innovation, especially for functions not directly related to the car itself but to the humans inside (audio, communication, and so on).

The industry is faced with different innovation rates for functions offered outside the car and there's a growing need to rapidly introduce innovations such as digital audio, smart phones, and social media. This



requires not just a way to upgrade the systems during the car's lifetime, but also a rapid response to new concepts introduced by the smart phone industry. Traditional development processes for cars and their (safety-critical) components¹ no longer keep pace with the introduction of innovative concepts.

Traditional development cycles focused on timely delivery of high-quality products and sought the complete elimination of uncertainty, both in time and quality. Projects typically started with a thick and detailed product specification designed to meet customer requirements four years later, followed by a three-year development cycle, including one year of validation of the final product. The result was a high-quality product designed to survive seven years in the new car catalog with a 10-year life cycle for the car itself. Many new cars have infotainment equipment based on technology already a decade old in the consumer market—for example, cassette audio has only recently been phased out and MP3 introduced!

Introducing higher speeds of innovation without compromising quality requires a different look at how to deal with project uncertainty.

Dealing with Uncertainty

TomTom has adapted its processes in three areas to accommodate short lead times:

- no waiting,
- no waste, and
- dealing with uncertainty.

Many projects show a very detailed activity plan, with clear due dates for each activity. Managers often feel their project is under control if they can ensure their engineers have met their “committed” dates. But because of development projects’ intrinsic uncertainty, every committed date requires a buffer, often expressed as the difference between the best case and worst case.

It comes down to human nature that such committed dates are seldom successfully met.²

We characterize our approach as an agile way of working (<http://agilemanifesto.org>) with a good backlog of activities and clear reporting on progress aimed at a best-case target. We combine this with a consolidated buffer at the end of the critical path, which allows shorter lead times and a higher probability of meeting deadlines. Managers should steer on buffer consumption and allow their engineers to consume the buffer, although it's better to avoid it whenever possible.

Traditional development cycles often implement a waterfall process beginning with a defined specification that includes change control and documents the design before implementation and a final test period. Because it's hard to predict consumer needs three years out owing to rapidly changing customer expectations, an increasing amount of change requests leads to a lot of wasted effort.

We've substantially reduced waste by organizing the project in an agile manner, starting with the well-defined section of the specification and leaving detail prioritization to the executing team, while maintaining close cooperation with the car manufacturer. The organization should focus on making the best product in the market rather than executing the contract and specification. Much traditional documentation (design documents and so on) is hardly ever used again and can easily be replaced by a more compact form, giving an engineer a faster introduction to the subject.

Each project is a compromise between specification, quality, and time. As we mentioned, quality is very important in the automotive industry and car manufacturers prefer to err on the side of safety. Quality requirements are therefore higher than the average end user might expect. When a tier-one supplier (communicating and delivering directly to the car OEM) is involved, this supplier

will again raise the quality requirements to minimize any quality risks.

Because introducing a new car is a complicated project with many suppliers, the design end dates for each component must be absolutely respected. This implies a buffer before the committed end date and an increased lead-time. The only way to limit this buffer is by adding a buffer in the specification. A standard way to achieve this is to split the specification into “must-do” and “nice-to-have” elements leading to the required buffer. By implementing an end-user friendly way of upgrading a released system with new functionality, organizations can reduce the importance of postponing certain functionality to later releases.

Such techniques considerably helped TomTom achieve faster lead times without compromising quality. We expect that innovations in the consumer space will occur even faster and influence customer expectations for people-centric car functionality. Car manufacturers might therefore need to reconsider some of their own processes to deal with functions that have different innovation rates.

Sales Volume and Software Size

The size of the application software in TomTom Portable Navigation Devices (PNDs) has grown rapidly (see Figure 1). The continuous introduction of new features has led to a yearly code size growth of typically 25–30 percent. Sales volume of the PNDs grew even faster, allowing the required investments for all high-speed innovations. The figures for 2009/2010 are difficult to interpret: the automotive industry crisis of 2008–2010 influenced the sales volume, and significant refactoring of the code base in 2010 resulted in a 4 percent decrease in code size despite ever-continuing feature growth. In 2011, the “normal” growth of the code size has resumed again.

In comparison with previous Impact columns, some similarities and differences come to mind. An obvious comparison is the one on the Bosch software in automotive systems.¹ A similarity is that both Bosch and TomTom have a higher volume than the largest car OEM. The route each company took to achieve that higher volume, however, is completely different. A major difference is that Bosch software is safety critical and closely integrated with dedicated integrated circuits for the automotive industry. It thus follows a different development approach. TomTom software development looks more like that of mobile phones³ than safety-critical flight management software⁴ or medical equipment.⁵

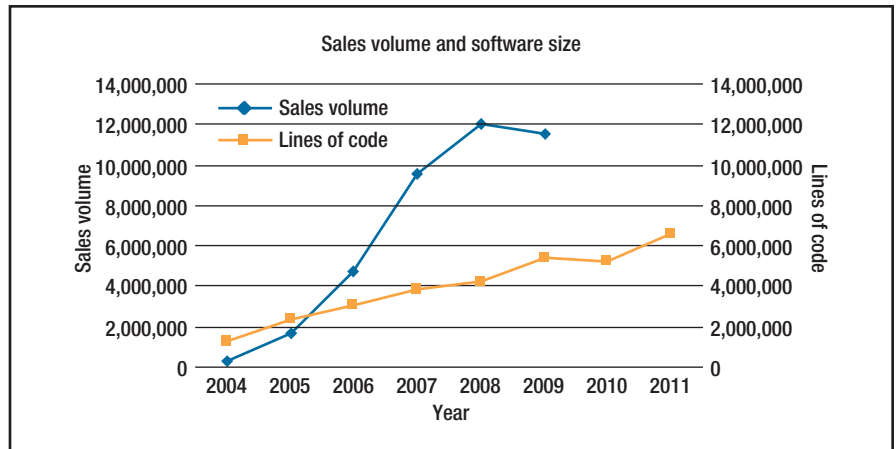


FIGURE 1. Growth of TomTom portable navigation device sales volume and code size over the years.

of the market leading to a shared number-two position in 2010. 

in MRI Scanners,” *IEEE Software*, vol. 27, no. 4, 2010, pp. 87–89.

TomTom has substantially increased the purchasing of in-dash navigation systems by some 400 percent because of the significantly lower price of entry. It achieved this lower price using existing consumer product technologies while meeting automotive quality requirements. From TomTom’s point of view, this has resulted in a much larger share

References

1. J. Mössinger, “Software in Automotive Systems,” *IEEE Software*, vol. 27, no. 2, 2010, pp. 92–94.
2. E. Goldratt, *Critical Chain*, North River Press, 1997.
3. L. Bouchard, “Multimedia Software for Mobile Phones,” *IEEE Software*, vol. 27, no. 3, 2010, pp. 8–10.
4. D. Avery, “The Evolution of Flight Management Systems,” *IEEE Software*, vol. 28, no. 1, 2011, pp. 11–13.
5. L. Hofland and J. van der Linden, “Software

HAN SCHAMINÉE is senior vice president of engineering at TomTom’s Business Unit Automotive. Contact him at han.schaminee@tomtom.com.

HANS AERTS is vice president of software development at TomTom’s Business Unit Automotive. Contact him at hans.aerts@tomtom.com.



Selected CS articles and columns are also available for free at <http://ComputingNow.computer.org>.

COMPUTING THEN

Learn about computing history and the people who shaped it.

<http://computingnow.computer.org/ct>